

Doublures de tests

Mockito

Pourquoi les doublures

- Les doublures sont des objets synthétiques qui remplacent un autre objet
- Les classes ont généralement des collaborateurs
- Ces collaborateurs sont parfois difficiles à instancier. Penser aux Repository de Spring par exemple.
- Des fois, que les interfaces sont fournies, les implémentations ne sont pas encore codés

Types de doublures

- Dummy objects: passés mais jamais utilisés
- Fake Objects: possèdent une implémentation qui est impropre à un environnement de production (ex: base de données en mémoire)
- Stub Objects: fournissent des réponses préprogrammées pour des cas précis
- Spy Objects: permettent d'observer les appels à un objet qu'ils remplacent
- Mock Objects: renferment une spécification des appels qui devront être reçus

Fake

- Une doublure Fake est un objet synthétique créer explicitement pour fins de tests. Il réimplante toute la fonctionnalité de l'objet qu'il remplace afin de satisfaire plusieurs scénarios de tests
- Un exemple serait une base de donnée en mémoire

Stub

- Le rôle du stub est de retourner des valeurs que l'on contrôle. On les décrits comme des entrées indirects pour fin de tests.
- Il existe 2 variantes
 - Responder: Fourni le data pour le 'happy day'
 - Saboteur: Fourni du data pour simuler des erreurs ou exceptions

Stub

```
public interface MailService {  
    public void send (Message msg);  
}  
  
public class MailServiceStub implements MailService {  
    private List<Message> messages = new ArrayList<Message>();  
  
    public void send(Message msg) {  
        messages.add(msg);  
    }  
  
    public int numberSent() {  
        return messages.size();  
    }  
}
```

Stub

```
class OrderStateTester {  
    @Test  
    public void testOrderSendsMailIfUnfilled() {  
        Order order = new Order("Time Out", 51);  
        MailServiceStub mailer = new MailServiceStub();  
        order.setMailer(mailer);  
        order.fill(warehouse);  
  
        assertEquals(1, mailer.numberSent());  
    }  
}
```

Spy

- Le 'spy' permet d'observer les appels à un objet qu'il observe
- Un 'spy' ne remplace pas un objet mais le décore afin d'observer son comportement

Mock

- Les objets mocks permettent de ‘vérifier’ le comportement d’un objet lors des tests. Un pas de plus que le ‘Stub’
- Cela permet de vérifier que les bonnes méthodes sont appelées ainsi que les bons arguments sont passés à la classe sous test
- Un des problèmes des Mock est qu’il met en évidence les détails d’implantations très rapidement. Faire attention

Mockito

```
<dependencies>  
  <dependency>  
    <groupId>org.mockito</groupId>  
    <artifactId>mockito-all</artifactId>  
    <version>1.9.5</version>  
    <scope>test</scope>  
  </dependency>  
  
  ...  
</dependencies>
```

Exemple de Mock